

[Algorithm]

탐색, 정렬 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



인하대학교
INHA UNIVERSITY

❖ 순차 탐색 (Sequential Search)

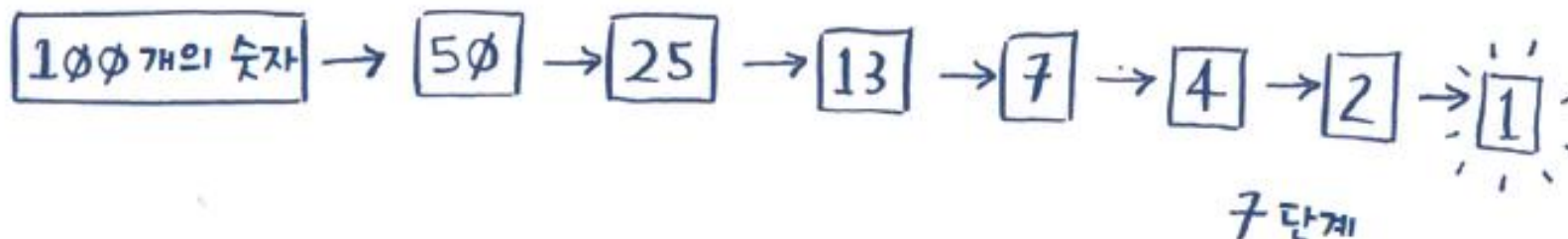
- 탐색 알고리즘
- 원소들이 정렬되어 있지 않을 경우에 원소들을 처음부터 비교하여 찾는 것임
- **$O(n)$ 알고리즘**
 - 순차 탐색은 배열에 있는 특정한 원소를 찾기 위하여 배열의 처음 원소부터 차례로 모든 원소들을 비교하여 탐색함
 - 이 방법은 효율적이지는 않지만 효과적임
 - 모든 원소들을 조사하는 탐색을 **선형 탐색(linear search)**이라고 함

```
int seq_search (int array[ ], int search_num, int n)
/* n개의 원소를 가진 배열 array에서 search_num을 순서대로 찾아서
   있으면 그의 지수를 반환하고 없으면 -1을 반환한다 */
{
    int i;
    array[n] = search_num;
    for(i = 0; array[i] != search_num; i++);
    return ((i < n)?i : -1);
}
```

❑ 이진탐색 알고리즘

❖ 이진 탐색 (Binary Search)

- 탐색 알고리즘
- 정렬된 원소 리스트를 입력받고,
원하는 원소가 있으면 그 원소의 위치를, 아니면 null 반환
- 탐색 범위의 중간부터 시작하여 비교를 통해 절반의 숫자 제거
 - 매 단계마다 남은 범위의 가운데 수를 말하고 대답에 따라 절반 제거 반복
- Ex) 1~100 중 하나의 숫자를 추측하여 맞추는 경우
 - 1부터 시작하여 하나씩 검토하는 단순 탐색은 비효율적
 - 50부터 시작하여 단계마다 절반의 숫자 제거
 - 최대 7번 만에 정답에 도달



□ 이진탐색 알고리즘

■ $O(\log_2 n)$ 알고리즘

- 전화번호부에서 사람의 이름을 찾을 때, 먼저 이름이 있을 곳을 추측하여 찾는데, 그곳이 찾는 사람의 이름보다 앞쪽의 이름이면 뒤쪽을 다시 찾고, 뒤쪽의 이름이면 앞쪽을 다시 찾음
- 이런 식으로 이름을 찾을 때까지 반복함
- 이와 같이 원소들이 순서대로 정렬되어 있을 때는 처음부터 찾을 필요가 없이 중간의 원소와 비교하여 그보다 작을 때에는 그 원소의 왼쪽 원소들 중에서, 클 때는 오른쪽 원소 중에서 다시 같은 형식으로 찾으면 훨씬 시간을 절약하여 찾을 수 있음

■ Ex) 100개의 원소를 가진 리스트에 대한 단순 / 이진 탐색 시간 비교

- 10억 개로 늘어난다면?
- 이진 탐색과 단순 탐색의 실행 시간은 같은 비율로 증가하지 않음

	단순 탐색	이진 탐색
100개	100밀리 초	7밀리 초
10,000개	10초	14밀리 초
1,000,000,000개	11일	32밀리 초

이진탐색 알고리즘

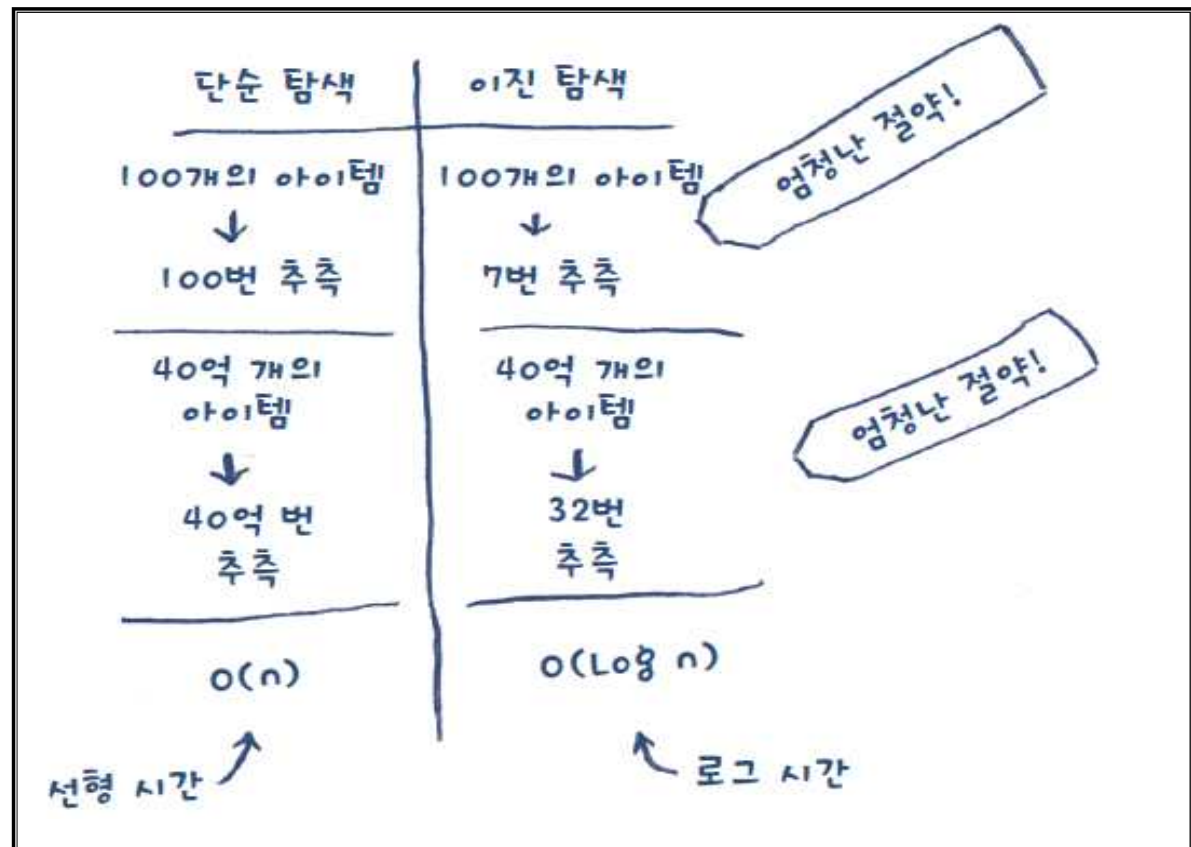
❖ 실행 시간 (Running Time)

■ 선형 시간 (Linear Time)

- 추측해야 할 최대 횟수는 리스트의 길이와 같음

■ 로그 시간 (Logarithmic Time)

- Ex) 이진 탐색



이진탐색 알고리즘



예제 12-8

배열의 원소들이 다음과 같은 순서를 가진다고 한다(5, 8, 11, 20, 25, 33, 48, 50, 57). 이 경우 찾고자 하는 원소가 11일 때 이진 탐색 알고리즘을 이용하여 수행 횟수에 따른 left와 right 값이 변하는 과정을 살펴보자.

풀이

수행 횟수	left	right
초기값	0	8
1	0	3
2	2	3

- 이진 탐색 알고리즘을 이용하여 원소를 찾는 과정은 이진 탐색 트리(binary search tree)를 이용하면 간단히 표현함
- 이진 탐색 트리는 처음 배열의 중간 원소를 루트(root)로 놓임
- 각 서브 트리의 루트는 둘로 나누어진 부분 집합들의 중간 원소들로 이루어짐

이진탐색 알고리즘



예제 12-8

배열의 원소들이 다음과 같은 순서를 가진다고 한다(5, 8, 11, 20, 25, 33, 48, 50, 57). 이 경우 찾고자 하는 원소가 11일 때 이진 탐색 알고리즘을 이용하여 수행 횟수에 따른 left와 right 값이 변하는 과정을 살펴보자.

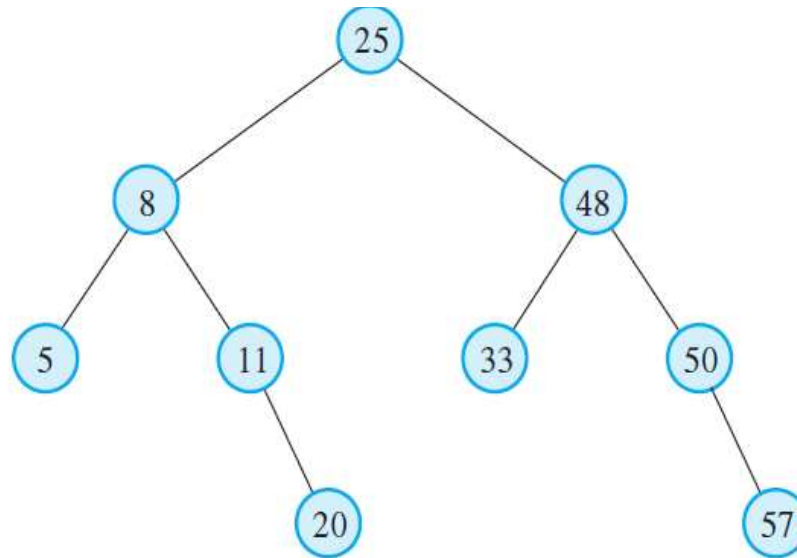
풀이

수행 횟수	left	right
초기값	0	8
1	0	3
2	2	3

- 이진 탐색 알고리즘을 이용하여 원소를 찾는 과정은 **이진 탐색 트리(binary search tree)**를 이용하면 간단히 표현함
- 이진 탐색 트리는 처음 배열의 중간 원소를 루트(root)로 놓임
- 각 서브 트리의 루트는 둘로 나뉘어진 부분 집합들의 중간 원소들로 이루어짐

이진탐색 알고리즘

이진 탐색 트리



- 이진 탐색 트리의 루트로 부터 트리의 노드로 가는 경로는 이진 탐색 알고리즘의 비교 횟수와 동일하므로 최악의 경우 트리의 깊이(depth)에 해당되는 비교 횟수인 $O(\log_2 n)$ 이 됨
- $O(\log_2 n)$ 인 이진 탐색은 $O(n)$ 인 순차 탐색보다 빠르고 효율적임
- 단, 이진 탐색을 위해서는 정렬이 되어 있어야 함

❖ 정렬 (Sort)

- 임의로 나열되어 있는 데이터들을 주어진 항목에 따라 크기 순서대로 작은 순서부터(**오름차순, ascending order**) 또는 큰 순서부터(**내림차순, descending order**) 늘어놓는 것임
- 정렬되어 있는 데이터들은 다음과 같은 작업을 수행할 때 응용순차 탐색은 배열에 있는 특정한 원소를 찾기 위하여 배열의 처음 원소부터 차례로 모든 원소들을 비교하여 탐색함

(1) 데이터를 탐색할 때

(2) 리스트(list)에 있는 다른 항목들을 비교할 때

■ **$O(n^2)$ 알고리즘**

- $O(n^2)$ 알고리즘으로는 널리 알려진 **버블 정렬(bubble sort)**과 **삽입 정렬(insertion sort)** 두 가지가 있음

버블 정렬



정렬 알고리즘

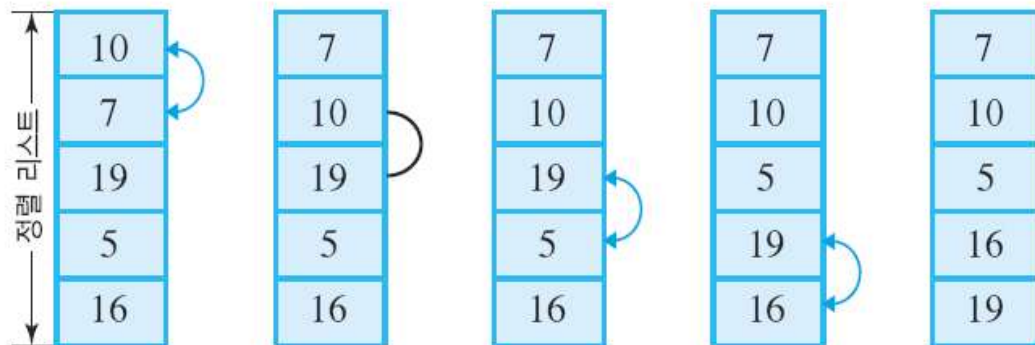


예제 12-9

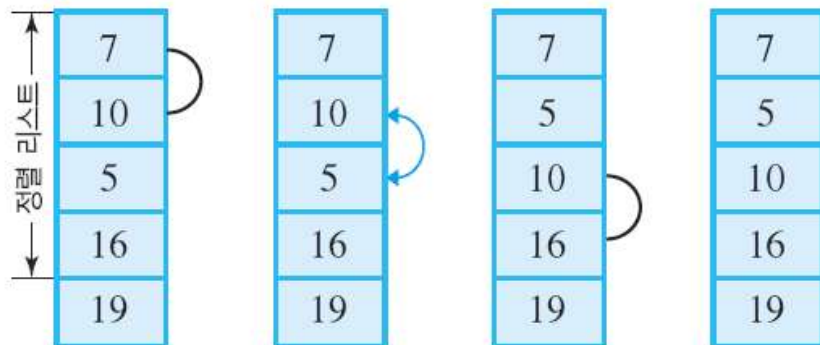
$n=5$ 이고 원소가 10, 7, 19, 5, 16일 때, 버블 정렬 알고리즘을 이용하여 수행하는 단계를 나타내어보자.

풀이 버블 정렬은 다음과 같은 네 단계를 거쳐 정렬될 수 있다. 그림에서 화살표가 있는 경우에는 교환을 하고, 화살표가 없는 경우에는 교환할 필요가 없으므로 비교만 나타낸다.

[1단계]

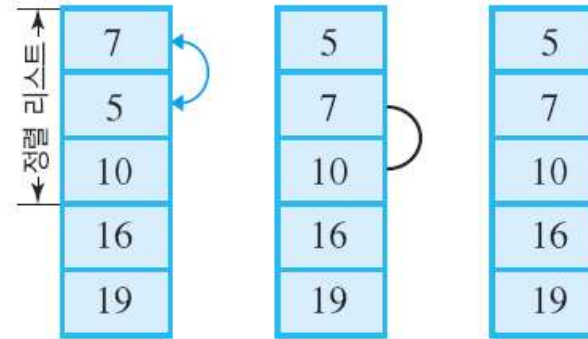


[2단계]

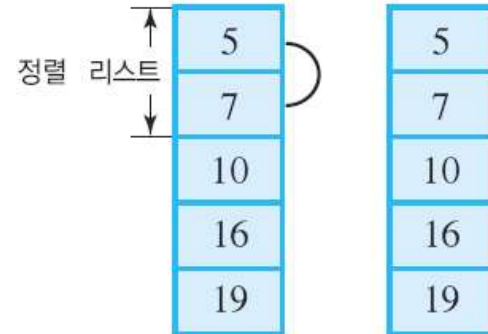


정렬 알고리즘

[3단계]



[4단계]



그 결과 크기가 큰 숫자가 가장 밑에서부터 차례대로 정렬되는 것을 알 수 있다.

```
void bubble_sort (int array[ ], int n)
/* 데이터들을 버블 정렬을 이용하여 오름차순으로 정렬한다 */
{
    int i, j, temp;
    for (i = 0; i < n-1; i++)
        for (j = 0; j < n-i-1; j++)
            if (array[j] > array[j+1]) {
                temp = array[j];
                array[j] = array[j+1];
                array[j+1] = temp;
            }
}
```

[프로그램 12.3] 버블 정렬 알고리즘

□ 정렬 알고리즘

- **버블 정렬(Bubble sort)**은 이웃한 두 개의 원소를 비교하여 순서가 서로 다르면 원소의 자리를 서로 바꾸고, 그렇지 않으면 그 위치에 그대로 놓음
- 위의 알고리즘에서 i 가 한 번 수행될 때마다 제일 오른쪽 끝에서부터 원소 중 큰 순서대로 정렬이 됨
- 앞의 알고리즘에 대한 수행 시간을 알아보면, $i = 0$ 일 때 j 에 대한 반복문은 $n-1$ 번 수행되며, $i = 1$ 일 때 $n-2$ 번 수행됨
- **알고리즘의 전체 수행시간**

$$(n-1) + (n-2) + \cdots + 2 + 1 = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n = O(n^2)$$

□ 사각형 칸 만들기 빅오 표기법

❖ 여러 가지 빅오 실행 시간

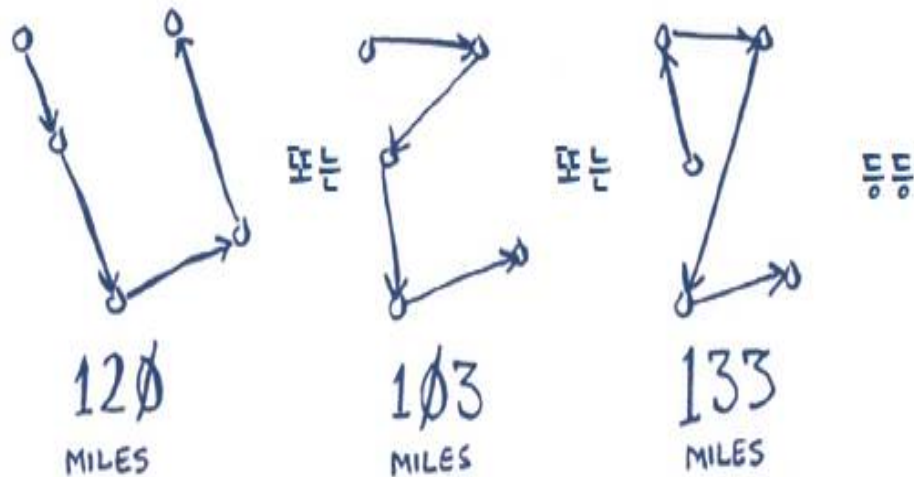
❖ 16개의 사각형 칸 생기도록 격자 만들기

- Ex 01) 한 번에 하나의 상자 그리는 경우
 - 16단계 필요
 - $O(n)$ 시간
- Ex 02) 종이를 한 번 접는 것이 하나의 연산인 경우
 - 한 번 접을 때마다 사각형의 수 두 배로 증가
 - 4단계 필요
 - $O(\log n)$ 시간

□ 외판원 문제 빅오 표기법

❖ 외판원 문제

- $O(n!)$ 시간 알고리즘
- 외판원이 다섯 개의 도시를 최단거리로 방문해야 하는 경우
 - 도시를 방문하는 모든 경로 구하기
 - 전체 거리 더하여 가장 짧은 경로 선택
 - n 개의 도시 있다면 $n!$ 번의 연산 필요

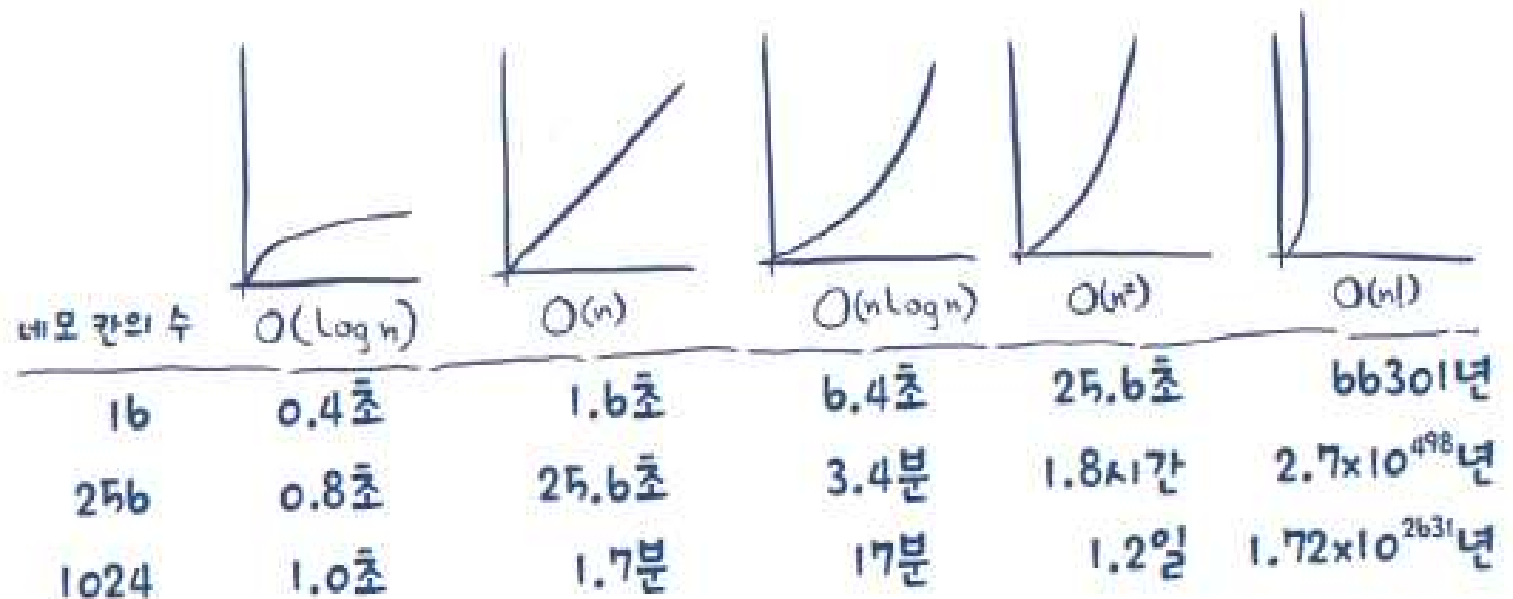


도시	연산
6	720
7	5040
8	40320
...	...
15	1,307,674,368,000
...	...
30	2,652,528,598,121,910,586,363,084,800,000,000

□ 시간함수 빅오 표기법 다시보기

❖ 많이 사용하는 빅오 실행 시간 예시

- $O(\log n)$
 - 이진 탐색
- $O(n)$
 - 단순 탐색
- $O(n \cdot \log n)$
- $O(n^2)$
- $O(n!)$





Thank You
